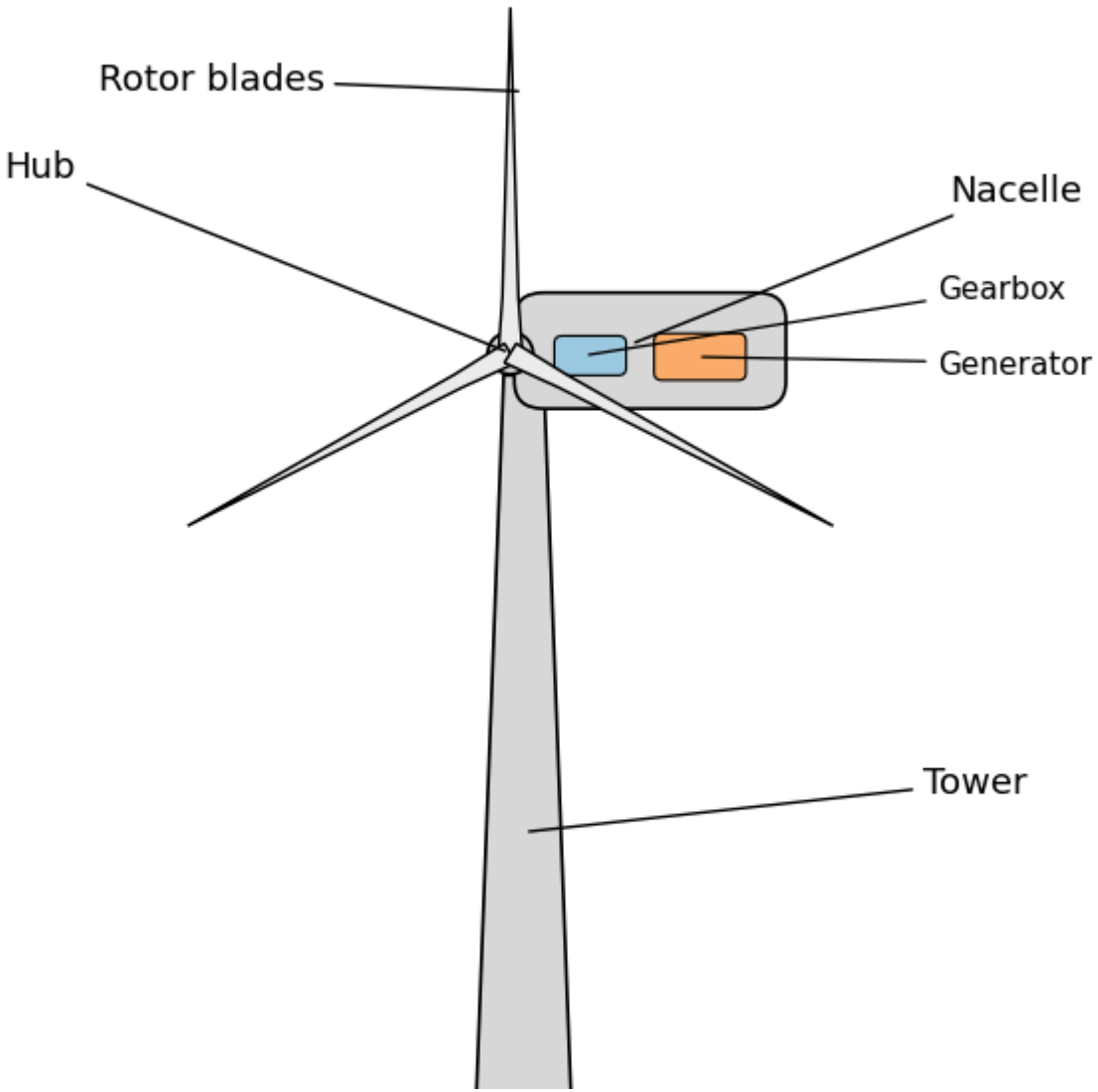
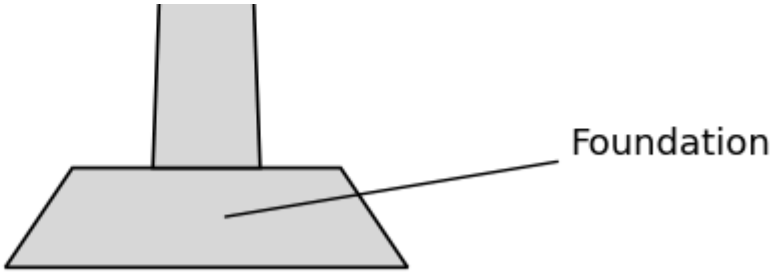
In [29]:

```python
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon, Circle, FancyBboxPatch
import numpy as np

fig, ax = plt.subplots(figsize=(7, 9))
ax.set_xlim(0, 10)
ax.set_ylim(0, 14)
ax.axis('off')

# Foundation
foundation = Polygon([[3.5, 0.8], [6.5, 0.8], [6.0, 1.6], [4.0, 1.6]],
                     closed=True, facecolor="#d9d9d9", edgecolor="black", linewidth=1.2)
ax.add_patch(foundation)

# Tower (shorter)
tower = Polygon([[4.6, 1.6], [5.4, 1.6], [5.15, 8.7], [4.85, 8.7]],
                closed=True, facecolor="#d9d9d9", edgecolor="black", linewidth=1.2)
ax.add_patch(tower)

# Nacelle
nacelle = FancyBboxPatch((4.95, 8.45), 2.0, 0.9,
                         boxstyle="round,pad=0.02,rounding_size=0.22",
                         facecolor="#d9d9d9", edgecolor="black", linewidth=1.2)
ax.add_patch(nacelle)

# Gearbox and generator
gearbox = FancyBboxPatch((5.25, 8.72), 0.5, 0.28,
                         boxstyle="round,pad=0.02,rounding_size=0.06",
                         facecolor="#9ecae1", edgecolor="black", linewidth=0.8)
generator = FancyBboxPatch((6.0, 8.68), 0.65, 0.34,
                           boxstyle="round,pad=0.02,rounding_size=0.06",
                           facecolor="#fdae6b", edgecolor="black", linewidth=0.8)
ax.add_patch(gearbox)
ax.add_patch(generator)

# Hub
hub_center = (4.9, 8.88)
hub = Circle(hub_center, 0.17, facecolor="#d9d9d9", edgecolor="black", linewidth=1.2)
ax.add_patch(hub)
```

```python
# Blade helper
def blade_polygon(length=2.8, width=0.18):
    return np.array([
        [0.0, -width/2],
        [0.45, -width/3],
        [length, 0.0],
        [0.45, width/3],
        [0.0, width/2]
    ])

def rotate_translate(points, angle_deg, origin):
    theta = np.deg2rad(angle_deg)
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta),  np.cos(theta)]])
    pts = points @ R.T
    pts[:, 0] += origin[0]
    pts[:, 1] += origin[1]
    return pts

# Larger blades
for angle in [90, 210, 330]:
    pts = rotate_translate(blade_polygon(), angle, hub_center)
    blade = Polygon(pts, closed=True, facecolor="#eeeeee", edgecolor="black", linewidth=1.0)
    ax.add_patch(blade)

# Labels
label_style = dict(
    arrowprops=dict(arrowstyle="-", lw=1.0, color="black"),
    fontsize=13,
    va="center"
)


ax.annotate("Rotor blades", xy=(5, 11), xytext=(1.8, 11.1), **label_style)
ax.annotate("Hub", xy=hub_center, xytext=(1.1, 10.4), **label_style)
ax.annotate("Nacelle", xy=(5.8, 8.95), xytext=(8.2, 10.2), **label_style)
ax.annotate("Tower", xy=(5.0, 5.0), xytext=(8.0, 5.4), **label_style)
ax.annotate("Foundation", xy=(5.1, 1.2), xytext=(7.7, 1.8), **label_style)

ax.annotate("Gearbox", xy=(5.45, 8.86), xytext=(8.1, 9.4),
```

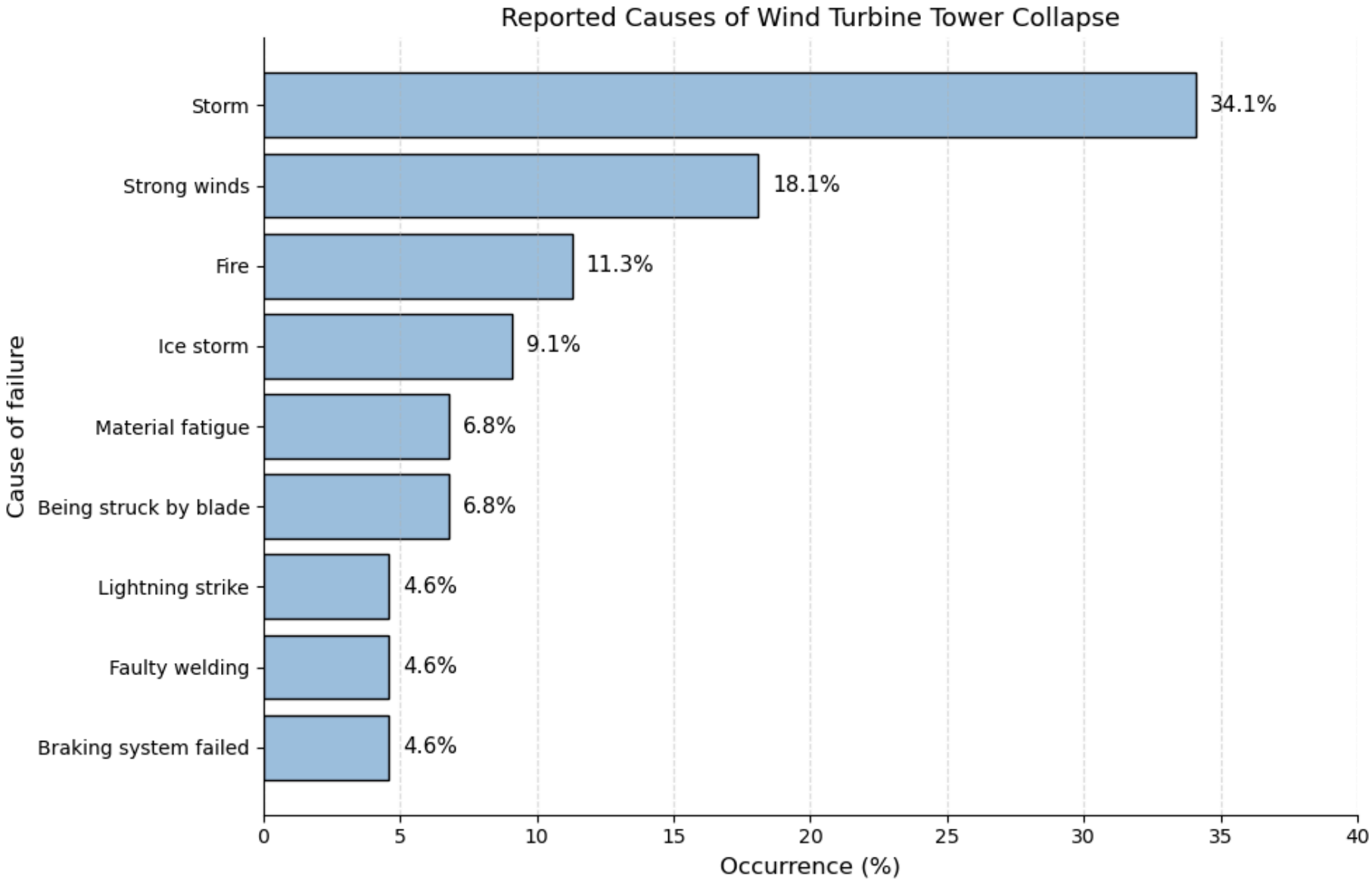
```
            arrowprops=dict(arrowstyle="-", lw=0.9, color="black"),
            fontsize=11, va="center")
ax.annotate("Generator", xy=(6.3, 8.85), xytext=(8.1, 8.8),
            arrowprops=dict(arrowstyle="-", lw=0.9, color="black"),
            fontsize=11, va="center")

plt.tight_layout()
plt.show()
```


Rotor blades
Hub
Nacelle
Gearbox
Generator
Tower



In [31]:
```
import matplotlib.pyplot as plt

# Data from Chou2011 Table 5
causes = [
    "Storm",
    "Strong winds",
    "Fire",
    "Ice storm",
    "Material fatigue",
    "Being struck by blade",
    "Lightning strike",
    "Faulty welding",
    "Braking system failed"
]

percentages = [34.1, 18.1, 11.3, 9.1, 6.8, 6.8, 4.6, 4.6, 4.6]

# Reverse order so the largest appears at the top
causes_plot = causes[::-1]
percentages_plot = percentages[::-1]

fig, ax = plt.subplots(figsize=(10, 6.5))

bars = ax.barh(causes_plot, percentages_plot, color="#9fbfdf", edgecolor="black")

# Add percentage labels
for bar, value in zip(bars, percentages_plot):
```

```python
    ax.text(value + 0.5, bar.get_y() + bar.get_height()/2,
            f"{value:.1f}%", va='center', fontsize=11)

ax.set_xlabel("Occurrence (%)", fontsize=12)
ax.set_ylabel("Cause of failure", fontsize=12)
ax.set_title("Reported Causes of Wind Turbine Tower Collapse", fontsize=13)
ax.set_xlim(0, 40)

# Clean style
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="x", linestyle="--", alpha=0.4)

plt.tight_layout()
plt.savefig("2.3_tower_collapse_causes.png", dpi=300, bbox_inches="tight")
plt.show()
```



In [57]:
```
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

# A4 portrait
fig, ax = plt.subplots(figsize=(8.27, 11.69))
```

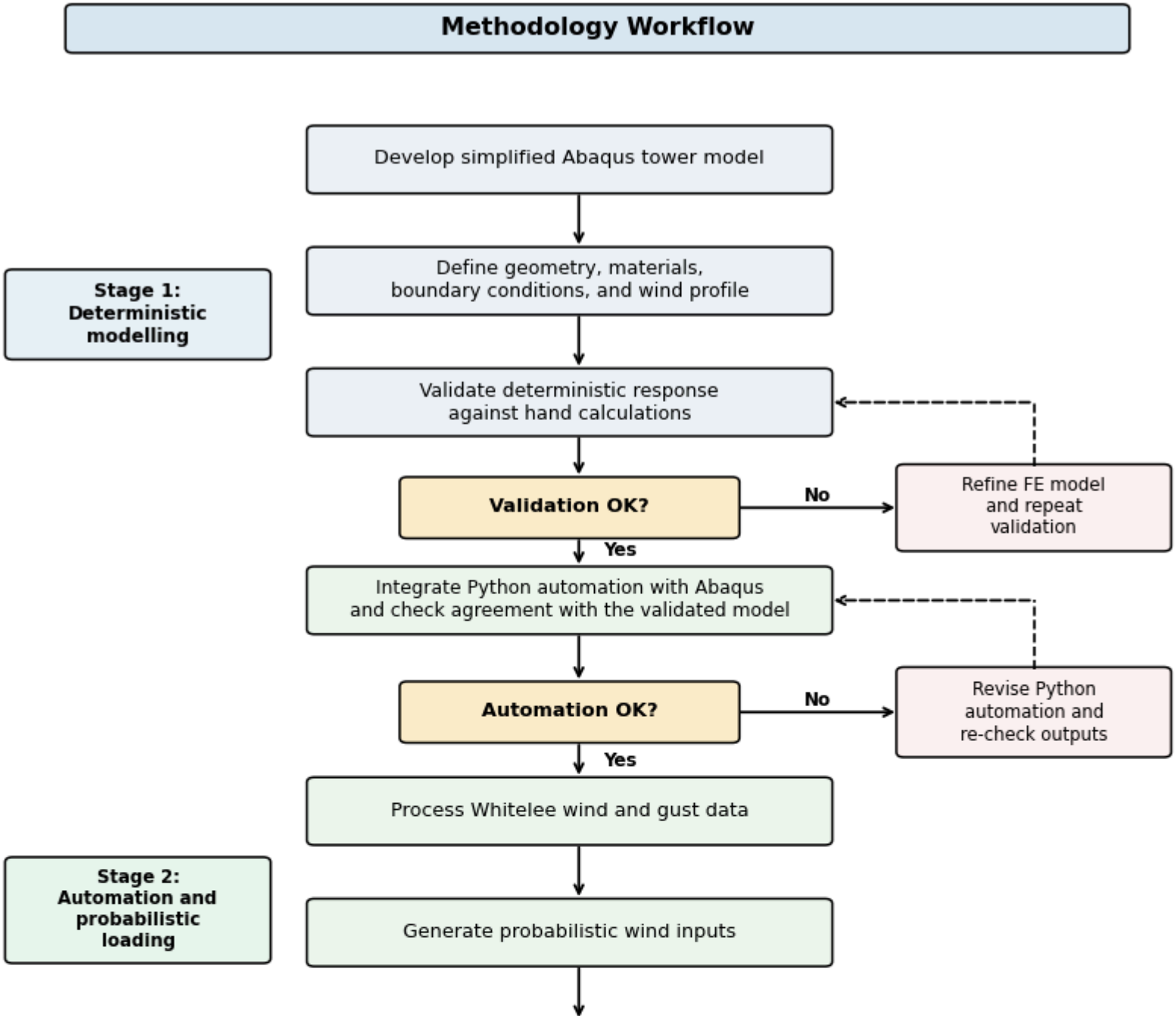
```python
ax.set_xlim(0, 13)
ax.set_ylim(-1.0, 26)
ax.axis("off")

# ---------- Helpers ----------
def add_box(x, y, w, h, text, fc="#f7f7f7", ec="black", fontsize=9.2, weight="normal", lw=1.1):
    box = FancyBboxPatch(
        (x, y), w, h,
        boxstyle="round,pad=0.03,rounding_size=0.08",
        linewidth=lw, edgecolor=ec, facecolor=fc
    )
    ax.add_patch(box)
    ax.text(
        x + w/2, y + h/2, text,
        ha="center", va="center",
        fontsize=fontsize, fontweight=weight, wrap=True
    )
    return {"x": x, "y": y, "w": w, "h": h}

def arrow(x1, y1, x2, y2, style="solid", connectionstyle="arc3"):
    linestyle = "-" if style == "solid" else "--"
    ax.annotate(
        "",
        xy=(x2, y2), xytext=(x1, y1),
        arrowprops=dict(
            arrowstyle="->",
            lw=1.3,
            color="black",
            linestyle=linestyle,
            connectionstyle=connectionstyle
        )
    )

def add_text(x, y, text, fontsize=8.5, weight="bold", ha="center", va="center"):
    ax.text(x, y, text, fontsize=fontsize, fontweight=weight, ha=ha, va=va)

def dashed_line(x1, y1, x2, y2):
    ax.plot([x1, x2], [y1, y2], linestyle="--", lw=1.2, color="black")

def dashed_return_from_top_to_right_mid(revision_box, target_box):
    sx = revision_box["x"] + revision_box["w"]/2
```

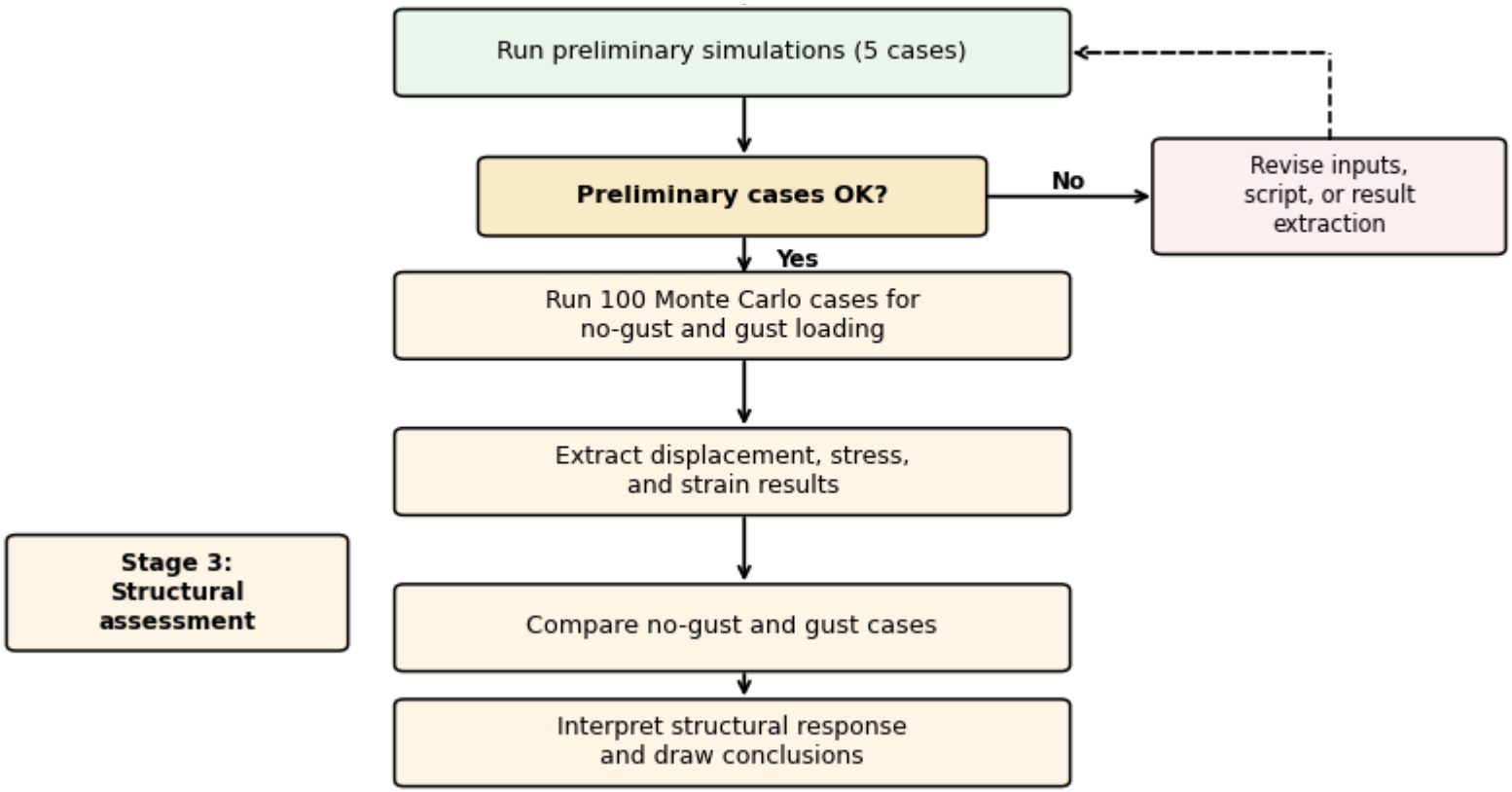
```
    sy = revision_box["y"] + revision_box["h"]

    tx = target_box["x"] + target_box["w"]
    ty = target_box["y"] + target_box["h"]/2

    # vertical segment from top-middle of pink box to the same y-level as target midpoint
    dashed_line(sx, sy, sx, ty)

    # horizontal arrow directly into the right-middle of the target box
    arrow(sx, ty, tx, ty, style="dashed")

# ---------- Layout ----------
cx = 6.3
main_x = 3.4
main_w = 5.6
main_h = 1.0

# Title
add_box(0.8, 24.8, 11.4, 0.7, "Methodology Workflow",
        fc="#d9e6f2", fontsize=11.5, weight="bold")

# Larger left stage boxes
add_box(0.15, 20.0, 2.8, 1.35, "Stage 1:\nDeterministic\nmodelling",
        fc="#eaf1f8", fontsize=8.8, weight="bold")
add_box(0.15, 10.55, 2.8, 1.6, "Stage 2:\nAutomation and\nprobabilistic\nloading",
        fc="#eaf7ef", fontsize=8.4, weight="bold")
add_box(0.15, 1.85, 2.8, 1.35, "Stage 3:\nStructural\nassessment",
        fc="#fff7ea", fontsize=8.8, weight="bold")

# Main workflow boxes
b1  = add_box(main_x, 22.6, main_w, main_h, "Develop simplified Abaqus tower model",
              fc="#eef3f8", fontsize=9.3)
b2  = add_box(main_x, 20.7, main_w, main_h, "Define geometry, materials,\nboundary conditions, and wind profile",
              fc="#eef3f8", fontsize=9.0)
b3  = add_box(main_x, 18.8, main_w, main_h, "Validate deterministic response\nagainst hand calculations",
              fc="#eef3f8", fontsize=9.0)

q1  = add_box(main_x + 1.0, 17.2, main_w - 2.0, 0.9, "Validation OK?",
              fc="#fdecc8", fontsize=9.2, weight="bold")

b4  = add_box(main_x, 15.7, main_w, main_h,
```

```
              "Integrate Python automation with Abaqus\nand check agreement with the validated model",
              fc="#eef8ef", fontsize=8.8)

q2  = add_box(main_x + 1.0, 14.0, main_w - 2.0, 0.9, "Automation OK?",
              fc="#fdecc8", fontsize=9.2, weight="bold")

b5  = add_box(main_x, 12.4, main_w, main_h, "Process Whitelee wind and gust data",
              fc="#eef8ef", fontsize=9.2)
b6  = add_box(main_x, 10.5, main_w, main_h, "Generate probabilistic wind inputs",
              fc="#eef8ef", fontsize=9.2)
b7  = add_box(main_x, 8.6, main_w, main_h, "Run preliminary simulations (5 cases)",
              fc="#eef8ef", fontsize=9.2)

q3  = add_box(main_x + 0.7, 6.9, main_w - 1.4, 0.9, "Preliminary cases OK?",
              fc="#fdecc8", fontsize=9.2, weight="bold")

b8  = add_box(main_x, 5.4, main_w, main_h, "Run 100 Monte Carlo cases for\nno-gust and gust loading",
              fc="#fff8ea", fontsize=9.0)
b9  = add_box(main_x, 3.5, main_w, main_h, "Extract displacement, stress,\nand strain results",
              fc="#fff8ea", fontsize=9.0)
b10 = add_box(main_x, 1.6, main_w, main_h, "Compare no-gust and gust cases",
              fc="#fff8ea", fontsize=9.2)
b11 = add_box(main_x, 0.2, main_w, main_h, "Interpret structural response\nand draw conclusions",
              fc="#fff8ea", fontsize=9.0)

# Revision boxes aligned with No lines
r_w = 2.9
r_h1 = 1.3
r_h2 = 1.35
r_x = 9.75

r1_y = (q1["y"] + q1["h"]/2) - r_h1/2
r2_y = (q2["y"] + q2["h"]/2) - r_h2/2
r3_y = (q3["y"] + q3["h"]/2) - r_h2/2

r1 = add_box(r_x, r1_y, r_w, r_h1, "Refine FE model\nand repeat\nvalidation",
             fc="#fdf0f0", fontsize=8.4)
r2 = add_box(r_x, r2_y, r_w, r_h2, "Revise Python\nautomation and\nre-check outputs",
             fc="#fdf0f0", fontsize=8.3)
r3 = add_box(r_x, r3_y, r_w, r_h2, "Revise inputs,\nscript, or result\nextraction",
             fc="#fdf0f0", fontsize=8.3)
```

```
# ---------- Main vertical arrows ----------
arrow(cx, 22.6, cx, 21.7)
arrow(cx, 20.7, cx, 19.8)
arrow(cx, 18.8, cx, 18.1)

arrow(cx, 17.2, cx, 16.7)
add_text(cx + 0.45, 16.98, "Yes")

arrow(cx, 15.7, cx, 14.9)

arrow(cx, 14.0, cx, 13.4)
add_text(cx + 0.45, 13.68, "Yes")

arrow(cx, 12.4, cx, 11.5)
arrow(cx, 10.5, cx, 9.6)
arrow(cx, 8.6, cx, 7.8)

arrow(cx, 6.9, cx, 6.35)
add_text(cx + 0.45, 6.58, "Yes")

arrow(cx, 5.4, cx, 4.5)
arrow(cx, 3.5, cx, 2.6)
arrow(cx, 1.6, cx, 1.2)

# ---------- No branches ----------
# Validation
q1_right_x = q1["x"] + q1["w"]
q1_yc = q1["y"] + q1["h"]/2
r1_left_x = r1["x"]

arrow(q1_right_x, q1_yc, r1_left_x, q1_yc)
add_text((q1_right_x + r1_left_x)/2, q1_yc + 0.18, "No")
dashed_return_from_top_to_right_mid(r1, b3)

# Automation
q2_right_x = q2["x"] + q2["w"]
q2_yc = q2["y"] + q2["h"]/2
r2_left_x = r2["x"]

arrow(q2_right_x, q2_yc, r2_left_x, q2_yc)
```

```
add_text((q2_right_x + r2_left_x)/2, q2_yc + 0.18, "No")
dashed_return_from_top_to_right_mid(r2, b4)

# Preliminary
q3_right_x = q3["x"] + q3["w"]
q3_yc = q3["y"] + q3["h"]/2
r3_left_x = r3["x"]

arrow(q3_right_x, q3_yc, r3_left_x, q3_yc)
add_text((q3_right_x + r3_left_x)/2, q3_yc + 0.18, "No")
dashed_return_from_top_to_right_mid(r3, b7)

plt.tight_layout()
plt.savefig("3.1_methodology_workflow.png", dpi=300, bbox_inches="tight")
plt.show()
```


Methodology Workflow
Develop simplified Abaqus tower model
Define geometry, materials, boundary conditions, and wind profile
Stage 1: Deterministic modelling
Validate deterministic response against hand calculations
Validation OK?
No
Refine FE model and repeat validation
Yes
Integrate Python automation with Abaqus and check agreement with the validated model
Automation OK?
No
Revise Python automation and re-check outputs
Yes
Process Whitelee wind and gust data
Stage 2: Automation and probabilistic loading
Generate probabilistic wind inputs

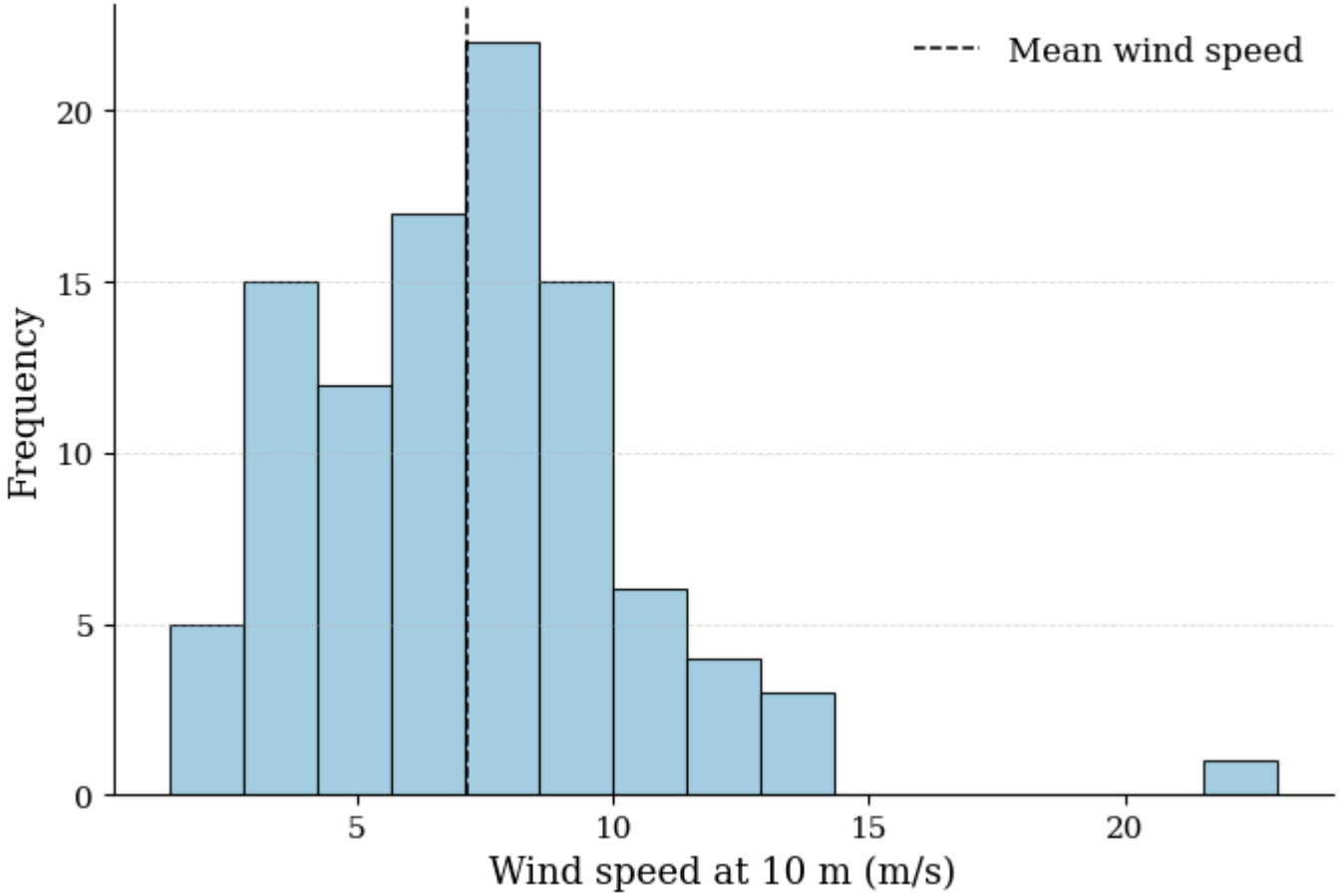


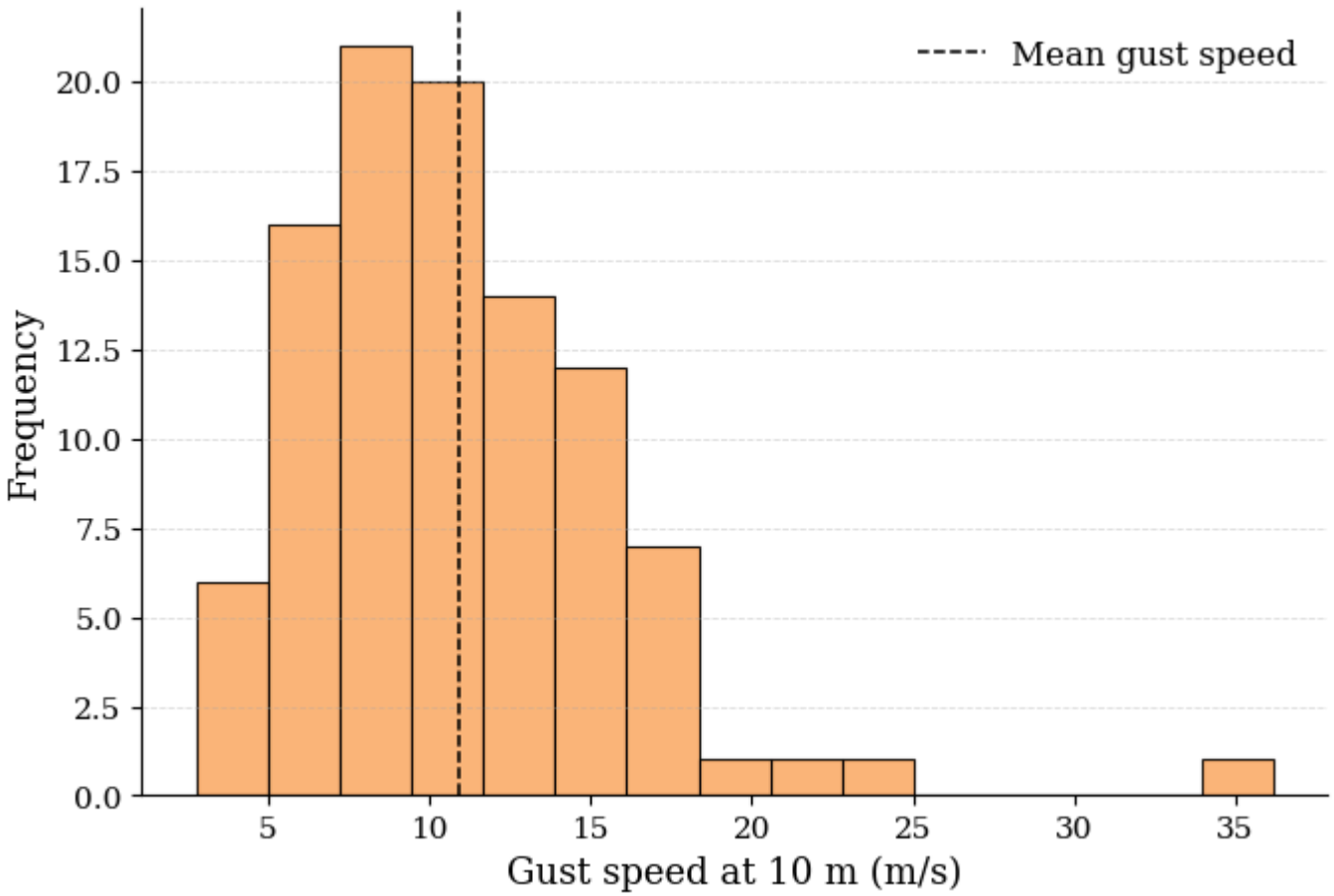
In [25]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# ----------------------------
# File paths
# ----------------------------
no_gust_file = "/Users/tanusrikopparthi/Downloads/tapered_no_gust_100.xlsx"
gust_file = "/Users/tanusrikopparthi/Downloads/tapered_all_gust_100.xlsx"

# ----------------------------
# Read Excel sheets
```

```python
# ------------------------------
df_wind = pd.read_excel(no_gust_file, sheet_name="wind_10m")
df_gust = pd.read_excel(gust_file, sheet_name="gust_10m")

# ------------------------------
# Extract sampled values (already in m/s)
# ------------------------------
wind_ms = df_wind["Wind_Mean_10m_m/s"].dropna()
gust_ms = df_gust["Wind_Peak_10m_m/s"].dropna()

# Quick checks
print("Number of wind samples:", len(wind_ms))
print("Number of gust samples:", len(gust_ms))
print("Wind mean:", wind_ms.mean())
print("Gust mean:", gust_ms.mean())

# ------------------------------
# Plot style
# ------------------------------
plt.rcParams.update({
    "font.family": "serif",
    "font.size": 12,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

def plot_histogram(data, xlabel, color, filename, legend_label, bins=8):
    fig, ax = plt.subplots(figsize=(7.0, 4.8))

    ax.hist(
        data,
        bins=bins,
        color=color,
        edgecolor="black",
        linewidth=0.8,
        alpha=0.9
    )

    mean_val = np.mean(data)
    ax.axvline(
```

```python
        mean_val,
        color="black",
        linestyle="--",
        linewidth=1.1,
        label=legend_label
    )

    ax.set_xlabel(xlabel)
    ax.set_ylabel("Frequency")

    ax.grid(axis="y", linestyle="--", linewidth=0.6, alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.legend(frameon=False, loc="upper right")

    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.show()

# ----------------------------
# Generate plots
# ----------------------------
plot_histogram(
    wind_ms,
    xlabel="Wind speed at 10 m (m/s)",
    color="#9ecae1",
    filename="Mean_Wind_10m.png",
    legend_label="Mean wind speed",
    bins=15
)

plot_histogram(
    gust_ms,
    xlabel="Gust speed at 10 m (m/s)",
    color="#fdae6b",
    filename="Mean_Gust_10m.png",
    legend_label="Mean gust speed",
    bins=15
)
```

```
Number of wind samples: 100
Number of gust samples: 100
Wind mean: 7.141439674848601
Gust mean: 10.9060271838142
```

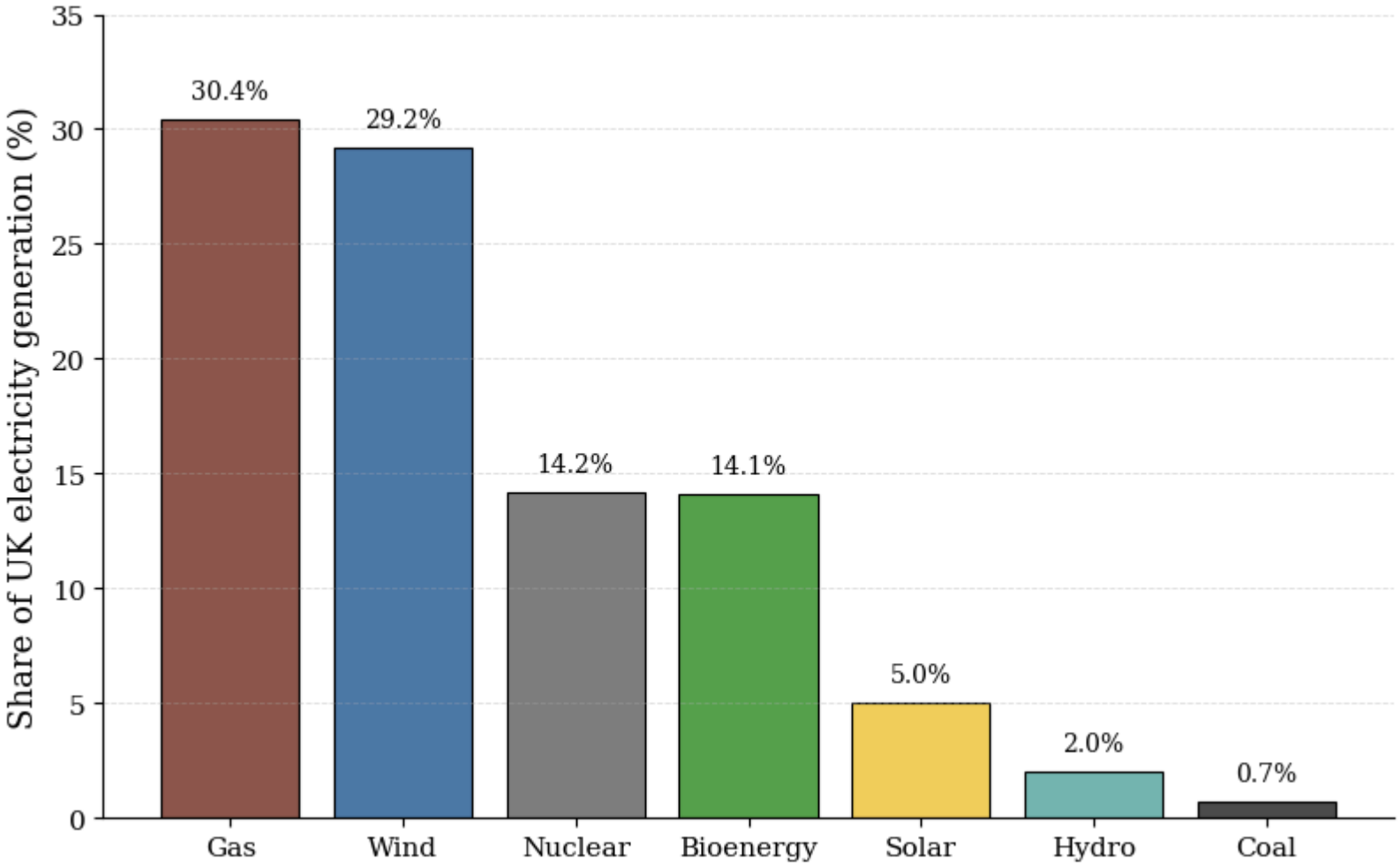




In [1]:
```
import matplotlib.pyplot as plt
import numpy as np

# Data from DUKES 2024 Chapter 5
sources = [
    "Gas",
    "Wind",
    "Nuclear",
    "Bioenergy",
    "Solar",
    "Hydro",
    "Coal"
```

```python
]

shares = [30.4, 29.2, 14.2, 14.1, 5.0, 2.0, 0.7]

# Plot style
plt.rcParams.update({
    "font.family": "serif",
    "font.size": 12,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

colors = [
    "#8c564b",  # Gas
    "#4C78A8",  # Wind
    "#7f7f7f",  # Nuclear
    "#59A14F",  # Bioenergy
    "#F2CF5B",  # Solar
    "#76B7B2",  # Hydro
    "#4d4d4d"   # Coal
]

fig, ax = plt.subplots(figsize=(8.2, 5.2))

bars = ax.bar(sources, shares, color=colors, edgecolor="black", linewidth=0.8)

# Add value labels
for bar, value in zip(bars, shares):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.7,
        f"{value:.1f}%",
        ha="center",
        va="bottom",
        fontsize=10
    )

ax.set_ylabel("Share of UK electricity generation (%)")
ax.set_ylim(0, 35)
```

```
ax.grid(axis="y", linestyle="--", linewidth=0.6, alpha=0.4)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

plt.tight_layout()
plt.savefig("1.1_uk_generation_mix.png", dpi=300, bbox_inches="tight")
plt.show()
```



In [51]:
```
import pandas as pd
import numpy as np
```

```python
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

# ---------------------------------------------------
# File paths and sheet names
# ---------------------------------------------------
NO_GUST_FILE = "/Users/tanusrikopparthi/Downloads/tapered_no_gust_100.xlsx"
GUST_FILE = "/Users/tanusrikopparthi/Downloads/tapered_all_gust_100.xlsx"

NO_GUST_SHEET = "NO_GUST"
GUST_SHEET = "ALL_GUST"

# ---------------------------------------------------
# Exact column names from your Excel sheets
# ---------------------------------------------------
HEIGHT_COL = "Height_m"
DISP_COL = "U_avg_Displacement_mm"
STRESS_COL = "S_avg_VonMises_Stress_MPa"
STRAIN_COL = "Strain_avg"

CASE_CANDIDATES = [
    "Case_ID", "CaseID", "Case", "case_id", "caseid", "case"
]

# ---------------------------------------------------
# Plot styling
# ---------------------------------------------------
plt.rcParams.update({
    "font.family": "serif",
    "font.size": 12,
    "axes.labelsize": 13,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11
})

NO_GUST_COLOR = "#4C78A8"
GUST_COLOR = "#F58518"
BAND_ALPHA = 0.12
SAMPLE_ALPHA = 0.25
SAMPLE_LW = 1.0
```

```python
MEAN_LW = 2.2
REP_ALPHA = 0.35
REP_LW = 1.0

# ---------------------------------------------------
# Detect case column automatically
# ---------------------------------------------------
def detect_case_column(df):
    for col in CASE_CANDIDATES:
        if col in df.columns:
            return col
    raise KeyError(
        "Could not find a case column. Available columns are:\n"
        + str(df.columns.tolist())
    )

# ---------------------------------------------------
# Load summary-style data for stress/strain
# ---------------------------------------------------
def load_results(file_path, sheet_name):
    df = pd.read_excel(file_path, sheet_name=sheet_name)
    df = df[[HEIGHT_COL, DISP_COL, STRESS_COL, STRAIN_COL]].copy()

    for col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    df = df.dropna(subset=[HEIGHT_COL, DISP_COL, STRESS_COL, STRAIN_COL])
    return df

# ---------------------------------------------------
# Summarise by height
# ---------------------------------------------------
def summarise_by_height(df):
    summary = (
        df.groupby(HEIGHT_COL, as_index=False)
          .agg({
              DISP_COL: ["mean", "std"],
              STRESS_COL: ["mean", "std"],
              STRAIN_COL: ["mean", "std"]
          })
    )
```

```python
    summary.columns = [
        "Height_m",
        "Disp_mean", "Disp_std",
        "Stress_mean", "Stress_std",
        "Strain_mean", "Strain_std"
    ]

    return summary.sort_values("Height_m").reset_index(drop=True)

# ---------------------------------------------------
# Load one response column for actual-sample selection
# ---------------------------------------------------
def load_response_results(file_path, sheet_name, response_col):
    df = pd.read_excel(file_path, sheet_name=sheet_name)

    case_col = detect_case_column(df)

    required_cols = [case_col, HEIGHT_COL, response_col]
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        raise KeyError(
            f"Missing required columns: {missing}\n"
            f"Available columns are:\n{df.columns.tolist()}"
        )

    df = df[required_cols].copy()

    df[case_col] = pd.to_numeric(df[case_col], errors="coerce")
    df[HEIGHT_COL] = pd.to_numeric(df[HEIGHT_COL], errors="coerce")
    df[response_col] = pd.to_numeric(df[response_col], errors="coerce")

    df = df.dropna(subset=[case_col, HEIGHT_COL, response_col])

    return df, case_col

# ---------------------------------------------------
# Build profile matrix: rows = height, cols = case
# ---------------------------------------------------
def build_profiles(df, case_col, response_col):
    profiles = df.pivot_table(
```

```python
        index=HEIGHT_COL,
        columns=case_col,
        values=response_col,
        aggfunc="mean"
    )

    profiles = profiles.sort_index()
    profiles = profiles.dropna(axis=1, how="any")

    return profiles

# ---------------------------------------------------
# Actual representative samples for displacement
# ---------------------------------------------------
def compute_case_scores(profiles, method="top"):
    if method == "top":
        top_height = profiles.index.max()
        return profiles.loc[top_height]
    elif method == "max":
        return profiles.max(axis=0)
    else:
        raise ValueError(f"Unknown method: {method}")

def compute_profile_rmse(profiles):
    mean_profile = profiles.mean(axis=1)
    diffs = profiles.sub(mean_profile, axis=0)
    rmse = np.sqrt((diffs ** 2).mean(axis=0))
    return rmse

def select_actual_representative_cases(profiles, method="top", initial_window_factor=0.30):
    case_scores = compute_case_scores(profiles, method=method)
    profile_rmse = compute_profile_rmse(profiles)

    mu = case_scores.mean()
    sigma = case_scores.std(ddof=1)

    if sigma > 0:
        targets = [
            mu - sigma,
            mu - 0.5 * sigma,
            mu,
```

```
            mu + 0.5 * sigma,
            mu + sigma
        ]
    else:
        targets = [mu] * min(5, len(case_scores))

    used_cases = set()
    selected_cases = []

    if sigma == 0:
        ordered = profile_rmse.sort_values().index.tolist()
        selected_cases = ordered[:min(5, len(ordered))]
        return selected_cases

    for target in targets:
        window = initial_window_factor * sigma
        candidates = []

        while len(candidates) == 0:
            candidates = [
                c for c in case_scores.index
                if c not in used_cases and abs(case_scores[c] - target) <= window
            ]
            window *= 1.5
            if window > 3.0 * sigma:
                break

        if len(candidates) == 0:
            candidates = [c for c in case_scores.index if c not in used_cases]

        best_case = min(
            candidates,
            key=lambda c: (profile_rmse[c], abs(case_scores[c] - target))
        )

        selected_cases.append(best_case)
        used_cases.add(best_case)

    selected_cases = sorted(selected_cases, key=lambda c: case_scores[c])
    return selected_cases
```

```python
# ---------------------------------------------------
# Generate smoother representative profiles for stress/strain
# ---------------------------------------------------
def generate_representative_profiles(mean_vals, std_vals, n_profiles=5, seed=42, nonnegative=True):
    rng = np.random.default_rng(seed)
    n = len(mean_vals)

    base_levels = np.array([-0.8, -0.4, 0.0, 0.4, 0.8])
    profiles = []

    for i in range(n_profiles):
        base = base_levels[i]
        raw = rng.normal(0, 0.18, size=n)

        smooth = raw.copy()
        for _ in range(4):
            smooth[1:-1] = (
                0.25 * smooth[:-2]
                + 0.5 * smooth[1:-1]
                + 0.25 * smooth[2:]
            )

        offset = base + smooth
        offset = np.clip(offset, -0.95, 0.95)

        prof = mean_vals + offset * std_vals

        if nonnegative:
            prof = np.maximum(prof, 0)

        profiles.append(prof)

    return profiles

# ---------------------------------------------------
# Common plotting utility
# ---------------------------------------------------
def get_common_limits(y_no, mean_no, std_no, extra_no,
                      y_g, mean_g, std_g, extra_g,
                      nonnegative=True):
    lower_no = mean_no - std_no
```

```python
    upper_no = mean_no + std_no
    lower_g = mean_g - std_g
    upper_g = mean_g + std_g

    if nonnegative:
        lower_no = np.maximum(lower_no, 0)
        lower_g = np.maximum(lower_g, 0)

    all_x = [lower_no.min(), upper_no.max(), lower_g.min(), upper_g.max()]

    for vals in extra_no:
        all_x.extend([np.min(vals), np.max(vals)])
    for vals in extra_g:
        all_x.extend([np.min(vals), np.max(vals)])

    x_min = min(all_x)
    x_max = max(all_x)

    if nonnegative:
        x_min = max(0, x_min)

    x_pad = 0.05 * (x_max - x_min) if x_max > x_min else 1.0
    x_min -= x_pad
    x_max += x_pad

    if nonnegative:
        x_min = max(0, x_min)

    y_max = max(np.max(y_no), np.max(y_g))
    return x_min, x_max, y_max, lower_no, upper_no, lower_g, upper_g

# ---------------------------------------------------
# Plot displacement with ACTUAL representative samples
# ---------------------------------------------------
def plot_displacement_with_actual_samples(
    no_profiles,
    gust_profiles,
    filename
):
    no_summary = pd.DataFrame({
        "Height_m": no_profiles.index.values,
```

```
        "Mean": no_profiles.mean(axis=1).values,
        "Std": no_profiles.std(axis=1, ddof=1).fillna(0).values
    })

    gust_summary = pd.DataFrame({
        "Height_m": gust_profiles.index.values,
        "Mean": gust_profiles.mean(axis=1).values,
        "Std": gust_profiles.std(axis=1, ddof=1).fillna(0).values
    })

    no_cases = select_actual_representative_cases(no_profiles, method="top")
    gust_cases = select_actual_representative_cases(gust_profiles, method="top")

    y_no = no_summary["Height_m"].values
    m_no = no_summary["Mean"].values
    s_no = no_summary["Std"].values

    y_g = gust_summary["Height_m"].values
    m_g = gust_summary["Mean"].values
    s_g = gust_summary["Std"].values

    extra_no = [no_profiles[c].values for c in no_cases]
    extra_g = [gust_profiles[c].values for c in gust_cases]

    x_min, x_max, y_max, lower_no, upper_no, lower_g, upper_g = get_common_limits(
        y_no, m_no, s_no, extra_no,
        y_g, m_g, s_g, extra_g,
        nonnegative=True
    )

    fig, axes = plt.subplots(1, 2, figsize=(12.5, 7.2), sharey=True)

    # No gust
    ax = axes[0]
    ax.fill_betweenx(y_no, lower_no, upper_no, color=NO_GUST_COLOR, alpha=BAND_ALPHA)

    for case in no_cases:
        ax.plot(
            no_profiles[case].values,
            no_profiles.index.values,
            color=NO_GUST_COLOR,
```

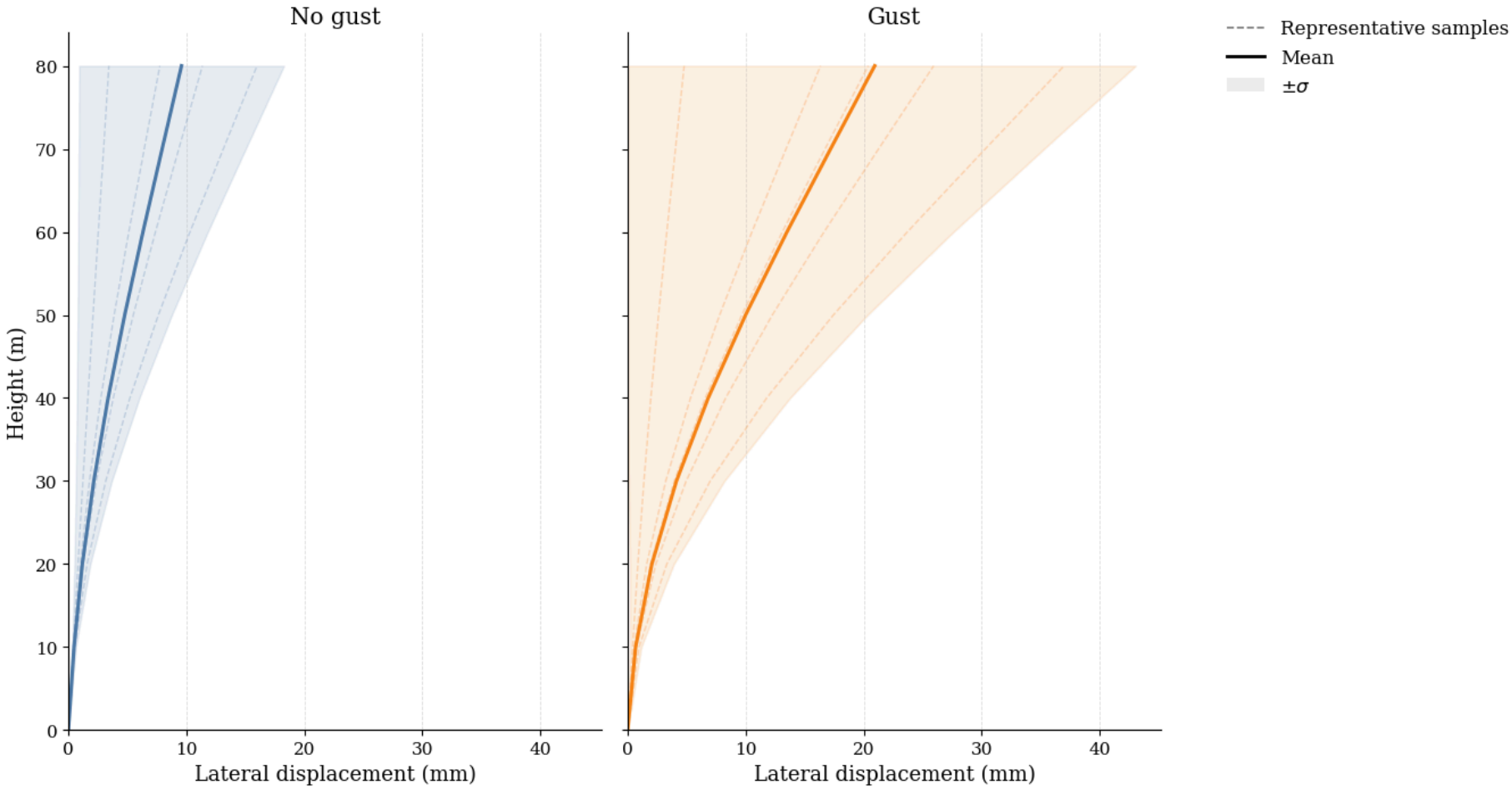
```python
            linestyle="--",
            linewidth=SAMPLE_LW,
            alpha=SAMPLE_ALPHA
        )

    ax.plot(m_no, y_no, color=NO_GUST_COLOR, linewidth=MEAN_LW)
    ax.set_title("No gust")
    ax.set_xlabel("Lateral displacement (mm)")
    ax.set_ylabel("Height (m)")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(0, y_max + 4)
    ax.grid(axis="x", linestyle="--", linewidth=0.6, alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

    # Gust
    ax = axes[1]
    ax.fill_betweenx(y_g, lower_g, upper_g, color=GUST_COLOR, alpha=BAND_ALPHA)

    for case in gust_cases:
        ax.plot(
            gust_profiles[case].values,
            gust_profiles.index.values,
            color=GUST_COLOR,
            linestyle="--",
            linewidth=SAMPLE_LW,
            alpha=SAMPLE_ALPHA
        )

    ax.plot(m_g, y_g, color=GUST_COLOR, linewidth=MEAN_LW)
    ax.set_title("Gust")
    ax.set_xlabel("Lateral displacement (mm)")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(0, y_max + 4)
    ax.grid(axis="x", linestyle="--", linewidth=0.6, alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

    legend_handles = [
        Line2D([0], [0], color="grey", lw=1.2, linestyle="--", label="Representative samples"),
        Line2D([0], [0], color="black", lw=2.0, label="Mean"),
```

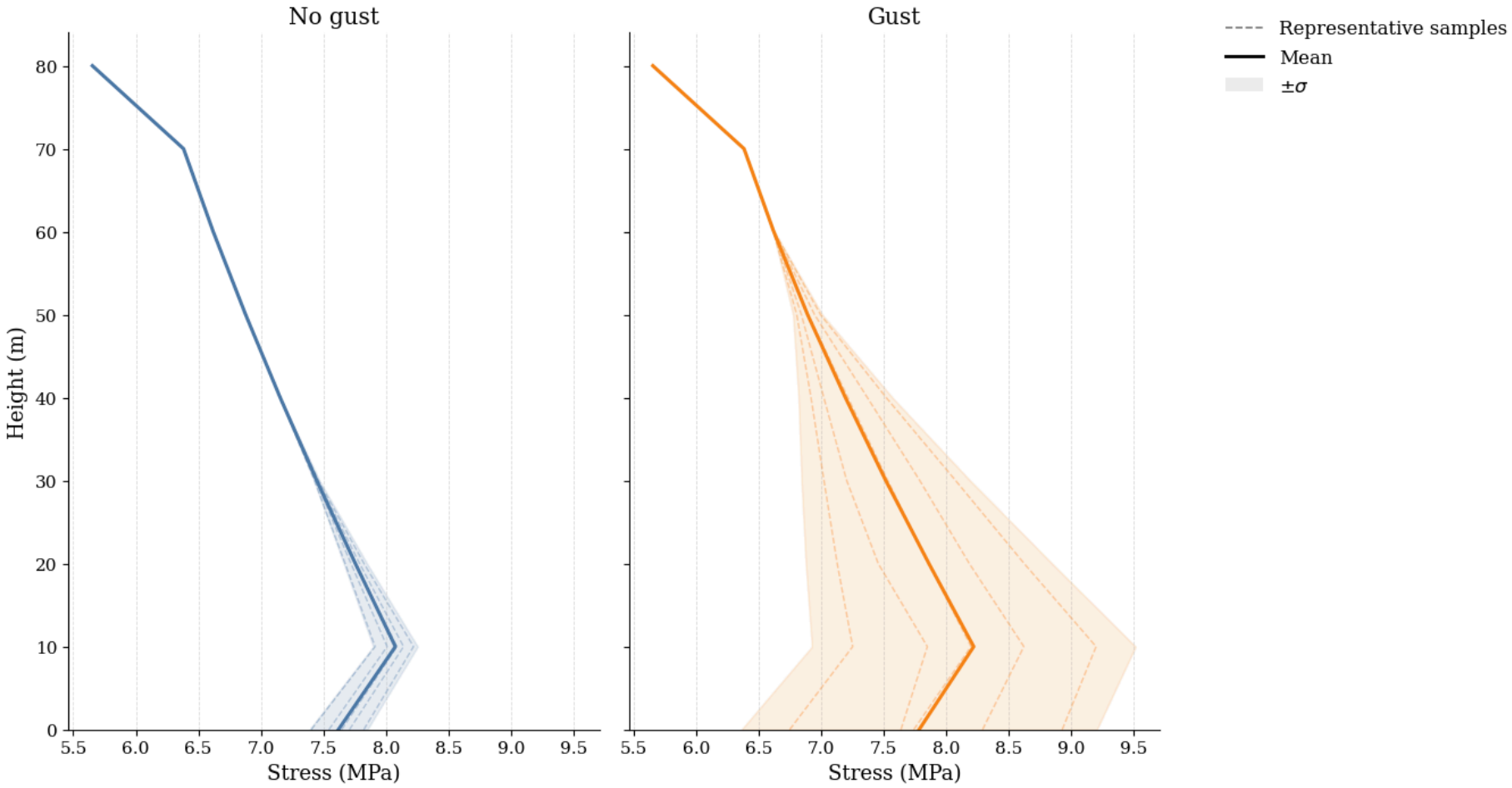
```python
        Patch(facecolor="grey", alpha=BAND_ALPHA, edgecolor="none", label=r"$\pm \sigma$")
    ]
    fig.legend(
        handles=legend_handles,
        frameon=False,
        loc="upper left",
        bbox_to_anchor=(0.86, 0.98)
    )

    plt.tight_layout(rect=[0, 0, 0.84, 1])
    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.show()

# ----------------------------------------------------
# Plot stress/strain with smoother representative samples
# ----------------------------------------------------
def plot_response_with_representative_samples(
    no_summary,
    gust_summary,
    mean_col,
    std_col,
    xlabel,
    filename,
    scale_factor=1.0,
    seed_no=123,
    seed_g=456
):
    y_no = no_summary["Height_m"].values
    m_no = no_summary[mean_col].values * scale_factor
    s_no = no_summary[std_col].fillna(0).values * scale_factor

    y_g = gust_summary["Height_m"].values
    m_g = gust_summary[mean_col].values * scale_factor
    s_g = gust_summary[std_col].fillna(0).values * scale_factor

    rep_no = generate_representative_profiles(m_no, s_no, n_profiles=5, seed=seed_no, nonnegative=True)
    rep_g = generate_representative_profiles(m_g, s_g, n_profiles=5, seed=seed_g, nonnegative=True)

    x_min, x_max, y_max, lower_no, upper_no, lower_g, upper_g = get_common_limits(
        y_no, m_no, s_no, rep_no,
        y_g, m_g, s_g, rep_g,
```

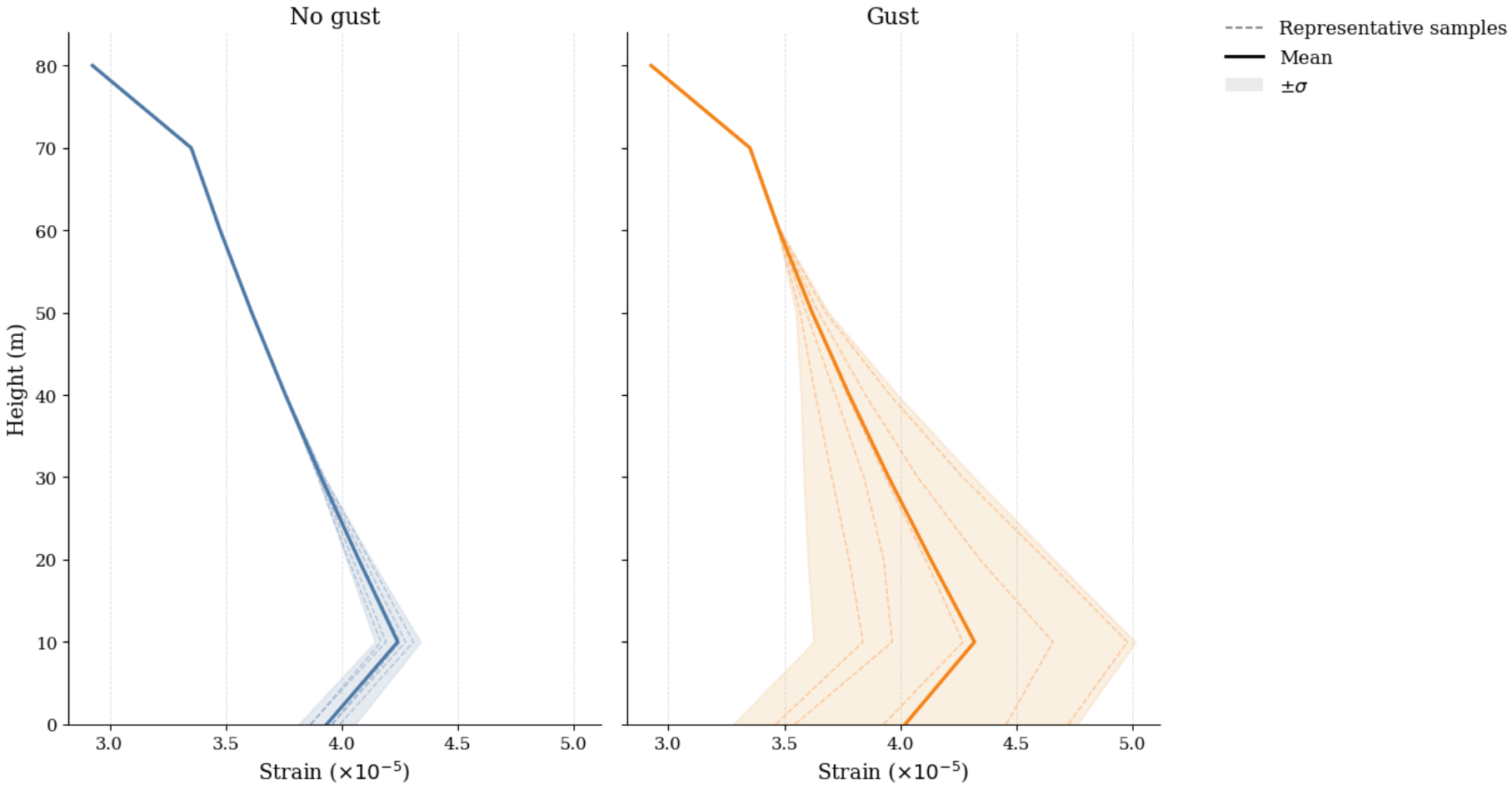
```
        nonnegative=True
    )

    fig, axes = plt.subplots(1, 2, figsize=(12.5, 7.2), sharey=True)

    # No gust
    ax = axes[0]
    ax.fill_betweenx(y_no, lower_no, upper_no, color=NO_GUST_COLOR, alpha=BAND_ALPHA)

    for vals in rep_no:
        ax.plot(vals, y_no, color=NO_GUST_COLOR, linestyle="--", linewidth=REP_LW, alpha=REP_ALPHA)

    ax.plot(m_no, y_no, color=NO_GUST_COLOR, linewidth=MEAN_LW)
    ax.set_title("No gust")
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Height (m)")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(0, y_max + 4)
    ax.grid(axis="x", linestyle="--", linewidth=0.6, alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

    # Gust
    ax = axes[1]
    ax.fill_betweenx(y_g, lower_g, upper_g, color=GUST_COLOR, alpha=BAND_ALPHA)

    for vals in rep_g:
        ax.plot(vals, y_g, color=GUST_COLOR, linestyle="--", linewidth=REP_LW, alpha=REP_ALPHA)

    ax.plot(m_g, y_g, color=GUST_COLOR, linewidth=MEAN_LW)
    ax.set_title("Gust")
    ax.set_xlabel(xlabel)
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(0, y_max + 4)
    ax.grid(axis="x", linestyle="--", linewidth=0.6, alpha=0.4)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

    legend_handles = [
        Line2D([0], [0], color="grey", lw=1.2, linestyle="--", label="Representative samples"),
        Line2D([0], [0], color="black", lw=2.0, label="Mean"),
```

```python
        Patch(facecolor="grey", alpha=BAND_ALPHA, edgecolor="none", label=r"$\pm \sigma$")
    ]
    fig.legend(
        handles=legend_handles,
        frameon=False,
        loc="upper left",
        bbox_to_anchor=(0.86, 0.98)
    )

    plt.tight_layout(rect=[0, 0, 0.84, 1])
    plt.savefig(filename, dpi=300, bbox_inches="tight")
    plt.show()

# ---------------------------------------------------
# Run
# ---------------------------------------------------
# Summary data
no_gust_df = load_results(NO_GUST_FILE, NO_GUST_SHEET)
gust_df = load_results(GUST_FILE, GUST_SHEET)

no_gust_summary = summarise_by_height(no_gust_df)
gust_summary = summarise_by_height(gust_df)

# Optional: save summaries
no_gust_summary.to_excel("no_gust_height_summary_with_std.xlsx", index=False)
gust_summary.to_excel("gust_height_summary_with_std.xlsx", index=False)

# Actual displacement profiles
no_disp_df, no_case_col = load_response_results(NO_GUST_FILE, NO_GUST_SHEET, DISP_COL)
gust_disp_df, gust_case_col = load_response_results(GUST_FILE, GUST_SHEET, DISP_COL)

no_disp_profiles = build_profiles(no_disp_df, no_case_col, DISP_COL)
gust_disp_profiles = build_profiles(gust_disp_df, gust_case_col, DISP_COL)

# Displacement: actual representative samples
plot_displacement_with_actual_samples(
    no_disp_profiles,
    gust_disp_profiles,
    filename="Displacement_vs_Height_sigma_samples_split.png"
)
```

```python
# Stress: smoother representative samples
plot_response_with_representative_samples(
    no_gust_summary,
    gust_summary,
    mean_col="Stress_mean",
    std_col="Stress_std",
    xlabel="Stress (MPa)",
    filename="Stress_vs_Height_sigma_samples_split.png",
    scale_factor=1.0,
    seed_no=303,
    seed_g=404
)

# Strain: smoother representative samples
plot_response_with_representative_samples(
    no_gust_summary,
    gust_summary,
    mean_col="Strain_mean",
    std_col="Strain_std",
    xlabel=r"Strain ($\times 10^{-5}$)",
    filename="Strain_vs_Height_sigma_samples_split.png",
    scale_factor=1e5,
    seed_no=505,
    seed_g=606
)
```


No gust
Gust
Representative samples
Mean
±σ
Height (m)
Lateral displacement (mm)
Lateral displacement (mm)
0
10
20
30
40
50
60
70
80
0
10
20
30
40
0
10
20
30
40


No gust
Gust
Height (m)
Stress (MPa)
Stress (MPa)
0
10
20
30
40
50
60
70
80
5.5
6.0
6.5
7.0
7.5
8.0
8.5
9.0
9.5
Representative samples
Mean
±σ


No gust
Gust
Representative samples
Mean
±σ
Height (m)
0
10
20
30
40
50
60
70
80
3.0
3.5
4.0
4.5
5.0
Strain (×10⁻⁵)

In [ ]: